VOLTCom: A Novel Online Trajectory Compression Method Based on Vector Processing

Zhi Cai, Qian Dong, Meihui Shi, Xing Su, Limin Guo, and Zhiming Ding

***Abstract*— With the widespread use of the Global Positioning System (GPS) in the fields such as traffic monitoring, sports navigation, and track recording, the trajectory data recording users' spatial and temporal information has grown dramatically. The huge volume of trajectory data causes high cost and poses a great challenge to data storage, network transmission, query and analysis. Therefore, the compression of trajectory data becomes a crucial issue. This paper proposes an online trajectory compression algorithm based on vector extraction (VOLTCom), which aims to achieve efficient data compression while retaining more effective information, and is mainly applied to trajectory recording and analysis in the traffic field. VOLTCom first generates vectors for trajectory data according to customized vector features, and then performs real-time vector extraction to achieve online trajectory compression. The vector extraction of the trajectory data ensures the stability of the compression time per unit and achieves efficient compression. Experiments on real datasets show that VOLTCom can retain the information of object velocity variation by vector density and outperforms traditional algorithms in terms of error, compression rate, and execution time. The algorithm is $O(1)$ in compression time complexity and has better compression performance.**

***Index Terms*— Online trajectory compression, vector, synchronous Euclidean distance, trajectory compression ratio.**

I. Introduction

With the development of technology [1], [2], [3], [4], the Global Positioning System (GPS) has been widely popularized in smart devices in recent years. For example, the location information of pedestrians can be collected by the GPS in smartphones; the location information of vehicles can be collected by vehicle GPS while they are moving, etc [5]. On the one hand, the real-time location data collected by GPS devices contain rich information that can be processed and used for trajectory recording and analysis in the transportation field, such as user behavior analysis [6], route planning [7], and urban traffic management scenarios. But on the other hand, the dramatic growth of trajectory data has brought a great challenge to their storage, transmission and analysis [8]. Statistics show that there are 67,000 taxis in Beijing. If each taxi sends location information to the data center every 2-3s, all taxis can generate 60TB of trajectory data every day [9].

To reduce the data scale of GPS trajectory data, the researchers have proposed various trajectory compression algorithms [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20]. The main idea is to retain the valuable key points of the original trajectory and control the error of the original trajectory and the compressed trajectory within a certain range. Classical algorithms, such as the Douglas-Peucker (DP) algorithm [13], the Top-Down Time Ratio (TD-TR) algorithm proposed by Meratnia et al. [14], the Sliding Window (SW) algorithm and the Opening Window (OPW) algorithm proposed by Keogh [15]. In addition, Spatial QUalIty Simplification Heuristic (SQUISH) algorithm [16] and improved SQUISH-E algorithm proposed by Jonathan Muckell et al. [17], as well as the road network-based trajectory compression algorithm combined with map matching techniques in recent years [5], among others.

Although the above algorithms can effectively eliminate trajectory redundant points to realize trajectory compression, they all store the compression results by saving the key trajectory points, which cannot reflect the temporal continuity of the compressed trajectory and increases the query workload of trajectory points. It also ignores the movement characteristics of speed changes, resulting in a certain loss of information. In addition, the implementation of most algorithms is relatively complex, and the simple algorithms, such as the DP algorithm has time-consuming operations, such as recursion, which greatly reduce the execution efficiency of the algorithm. In view of the above problems, the main contributions of this article are summarized as follows.

- We propose a Vector based OnLine Trajectory Compression algorithm (VOLTCom), which combines the vector extraction method for IoT sensing data [21] with the field of trajectory compression and simplifies and improves its original definition of vector features. The algorithm directly compresses the collected location data in real-time on the mobile terminal and stores the compressed trajectory in the form of vector sequence. It avoids the problems, such as traffic consumption and high storage overhead on the mobile terminal caused by offline background compression.
- We define a vector generation algorithm and a vector extraction algorithm. The vector generation algorithm uses Bézier function to process the trajectory data in

Manuscript received 20 January 2023; revised 23 May 2023; accepted 30 June 2023. Date of publication 28 July 2023; date of current version 29 November 2023. This work was supported in part by the National Natural Science of Foundation of China under Grant 62072016 and Grant 62276011 and in part by the Beijing Natural Science Foundation under Grant 4212016. The Associate Editor for this article was A. H. Sodhro. *(Corresponding author: Meihui Shi.)*

The authors are with the College of Computer Science, Beijing University of Technology, Beijing 100124, China (e-mail: caiz@bjut.edu.cn; dxinhaoq@163.com; shimeihui@bjut.edu.cn; xingsu@bjut.edu.cn; guolimin@bjut.edu.cn; zmding@bjut.edu.cn).

Digital Object Identifier 10.1109/TITS.2023.3294941

1558-0016 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

real-time to generate vector functions; the vector extraction algorithm constructs the generated vector function as a custom vector to generate a vector sequence of compressed trajectories to ensure continuity.

- We have conducted extensive experimental studies on several real data sets. Compared with the comparison algorithms, it is verified that the proposed algorithm not only has the characteristics of reflecting the velocity variation of the object by vector density and calculating the position information at an arbitrary time, but also has a shorter execution time, better compression efficiency and smaller error to some extent.
- We analyze and compare the time complexity of VOLTCom with other algorithms. The superiority of the linear time complexity of the proposed algorithm is theoretically discussed and verified by experiments.

The rest of the paper is organized as follows: SectionII reviews the related work. SectionIV gives the basic definitions required in this paper and describes in detail the steps of VOTCom. In SectionV, comparative experiments are performed and the experimental results are analyzed. In SectionVI, we analyze the time complexity of the typical algorithm and VOTCom from a theoretical point of view. Finally, SectionVII summarizes the algorithm and provides a preliminary discussion of the subsequent research.

II. The Related Work

In this section, the typical algorithms in the field of trajectory compression are divided into two categories: offline trajectory compression algorithms and online trajectory compression algorithms [22], and the related researches are intorduced.

The offline trajectory compression algorithms are based on the compression of the global trajectory points. Typical algorithms include the Douglas-Puker (DP) algorithm [13], which limits the maximum vertical Euclidean distance of the feature points to the line connecting the first and last nodes of the trajectory within a threshold, uses a recursive method to retain the feature points larger than the threshold, and divides the original trajectory into several approximate trajectory segments; Top-Down Time Ratio (TD-TR), a time-scaled top-down algorithm proposed by Meratnia et al. [14], improves DP by using the synchronized Euclidean distance to instead of the vertical Euclidean distance and adding a supplement to the time characteristics that is more comprehensive than the vertical Euclidean distance. Literature [18] proposed the DPhull algorithm based on DP, which uses convex packet structures and can obtain the same compressed trajectories with less time complexity. In addition, the SP algorithm and the approximate optimization algorithm are combined for the direction-preserving problem [19]. Although the offline trajectory compression algorithms have the global advantage and can guarantee the accuracy relative to the original trajectory, these algorithms are only applicable for the situation of clearly known historical trajectory data and not suitable for the real-time data acquisition systems. Moreover, the processing of global data by these types of algorithms reduces the efficiency of algorithm execution.

The online trajectory compression algorithms are based on the compression of local trajectory points which can avoid the problem of large consumption of spatial and temporal resources caused by the globality of the offline trajectory compression algorithm. This type of algorithms can upload the trajectory data to the server in real-time during the data collection process and compress the trajectory data before uploading. Typical algorithms include the Sliding Window(SW) algorithm and the Opening Window (OPW) algorithm proposed by Keogh et al. [15], which accomplish the simplification of the trajectory by adjusting the window size; later the Opening Window Time Ratio (OPW-TR) algorithm that replaces vertical Euclidean distance with synchronous Euclidean distance. The Spatial QUalIty Simplification Heuristic (SQUISH) algorithm [16] and the Spatial QUallty Simplification Heuristic-Extended (SQUISH-E) algorithm [17] proposed by Jonathan Muckell et al. are also online trajectory data compression algorithms. Among them, the SQUISH-E algorithm is special, it can set two parameters of compression ratio and synchronous Euclidean distance. When its compression ratio is set to 1, the algorithm works on its offline part [20]. Although these online trajectory compression algorithms can achieve local dominance, the global dominance is relatively weak and there is a greater risk of error, because they cannot take into account the overall trend of the data.

To this end, we propose VOLTCom in this paper. VOLTCom avoids the high time complexity problem caused by the global nature of the offline trajectory compression algorithm and the large error problem caused by the local nature of the online trajectory compression algorithm. Although VOLTCom is an online compression algorithm, the compression error is reduced by using a sequence of vector functions to fit the compressed trajectory. At the same time, there are many advantages, such as lower time complexity, etc.

III. Relevant Definition

The basic definitions related to the algorithm are presented and explained in detail in this section, where vector feature is the basic core definition of the algorithm in this paper.

Definition 1 (Trajectory): The trajectory point p_i can be defined as a triplet (lat_i, lng_i, t_i), where lat_i and lgn_i are the spatial position coordinates of p_i and t_i represents the time information generated by p_i, and the sequence formed according to the time order generated by the trajectory points is the trajectory $T = \{p_1, p_2, \cdots, p_n\}$, where n is the number of trajectory points of T. $\widehat{p_i p_j}$ denotes a line segment starting at p_i and ending at p_j.

Definition 2 (Vector Feature): The vector feature is expressed as $vector = \{f(t_i^R), t_{start}, t_{end}\}$, where t_{start} is the start time of the vector, t_{end} is the end time, $f(.)$ is a function related to time and the independent variable t_i^R is calculated as follows.

$$t_i^R = \frac{t_i - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}} \tag{1}$$

Although there are some other functions to fit the trajectory in the current situation, we consider that if a single linear

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

function is used to fit the trajectory, when the motion of the moving object is curved, the fitting of the line segment will bring a large error; if a single nonlinear function is used to fit, when the motion of the object is relatively simple and stable, it will bring unnecessary over-calculation, and the form of the nonlinear function is generally more complicated and not easy to understand. The first-order and second-order Bézier functions can not only represent the linear and curvilinear fits of the trajectory, but also have a simple expression form, which is easier to understand and calculate.

According to the algorithm requirements, we substitute t_i^R into the first or second-order Bézier function to find the specific value of $f(.)$, which are expressed as follows.

$$f(t_i^R) = (1 - t_i^R) * p_s + t_i^R * p_e \tag{2}$$

$$f(t_i^R) = (1 - t_i^R)^2 * p_s + 2 * t_i^R * (1 - t_i^R) * p_e + t_i^{R^2} * p_i, \tag{3}$$

where p_s and p_e are the spatial position coordinates corresponding to the previous vector features t_{start} and t_{end}, respectively, and p_i is the position information of the current received trajectory point.

From the core Equ. 2, when the motion state of the object is kept almost as uniform linear motion, a finite increasing error threshold can merge adjacent vector features and linearize the trajectory data, which can limit the time variation of the trajectory data.

Definition 3 (Compression Ratio): The compression ratio is the rate of the number of compressed trajectory points to the number of the original trajectory points. In this algorithm, the sum of the key starting and ending trajectory points used to extract the vector is calculated as the number of compressed trajectory points.

Definition 4 (Synchronous Euclidean Distance (sed)): The synchronous euclidean distance (sed) refers to the Euclidean distance between a point $P\,(lat, lng, t)$ on the original trajectory and the corresponding point $P'\left(lat', lng', t'\right)$ in the compressed trajectory according to the time scale. The calculation of sed is unified as follows.

$$sed = 2\text{arc}\sin\sqrt{\sin^2\left(\frac{a}{2}\right) + \cos\left(lat\right) * \cos\left(lat'\right) * \sin^2\left(\frac{b}{2}\right)} * R \tag{4}$$

where $a = lat - lat'$, $b = lng - lng'$, $R = 6,378,137$, and $sed\,(.)$ denotes the sed between the ith original trajectory point p_i and its calculated estimated point p_i'.

As shown in Fig. 1, $sed\left(\widehat{P_i P_i'}\right)$ and $sed\left(\widehat{P_j P_j'}\right)$ are the sed corresponding to the original trajectory points P_i and P_j, respectively.

For the calculation of the position coordinates of the corresponding points of the original trajectory points on the compressed trajectory, there are two cases as in Fig. 1,:

1) The compression trajectory is a straight line. Taking p_i as an example, substitute its spatiotemporal information into Equ.1 and Equ.2 to obtain the coordinates of its corresponding point coordinates, which is described as follows.

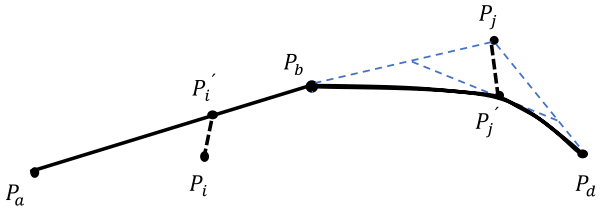


Fig. 1. Synchronized Euclidean distance and the second-order Bézier.

$$P_i' \cdot lat = \left(1 - \frac{t_i - t_a}{t_b - t_a}\right) * P_a \cdot lat + \frac{t_i - t_a}{t_b - t_a} * P_b \cdot lat \tag{5}$$

$$P_i' \cdot lng = \left(1 - \frac{t_i - t_a}{t_b - t_a}\right) * P_a \cdot lng + \frac{t_i - t_a}{t_b - t_a} * P_b.lng \tag{6}$$

2) The compression trajectory is a curve. Taking p_j as an example, substitute its spatiotemporal information into Equ.1 and Equ.3 to obtain the coordinates of its corresponding point coordinates, which is described as follows.

$$\begin{aligned} P_j' \cdot lat = &\left(1 - \frac{t_j - t_b}{t_d - t_b}\right)^2 * P_b \cdot lat \\ &+ 2 * \frac{t_j - t_b}{t_d - t_b} * \left(1 - \frac{t_j - t_b}{t_d - t_b}\right) * P_j \cdot lat \\ &+ \left(1 - \frac{t_j - t_b}{t_d - t_b}\right)^2 * P_d \cdot lat \end{aligned} \tag{7}$$

$$\begin{aligned} P_j' \cdot lng = &\left(1 - \frac{t_j - t_b}{t_d - t_b}\right)^2 * P_b \cdot lng \\ &+ 2 * \frac{t_j - t_b}{t_d - t_b} * \left(1 - \frac{t_j - t_b}{t_d - t_b}\right) * P_j \cdot lng \\ &+ \left(1 - \frac{t_j - t_b}{t_d - t_b}\right)^2 * P_d \cdot lng \end{aligned} \tag{8}$$

Definition 5 (Error): The error is measured using the average synchronous Euclidean distance $\overline{sed}$ which is calculated as follows.

$$\overline{sed} = \frac{\sum_{i=1}^{n} sed\left(\widehat{p_i p_i'}\right)}{n} \tag{9}$$

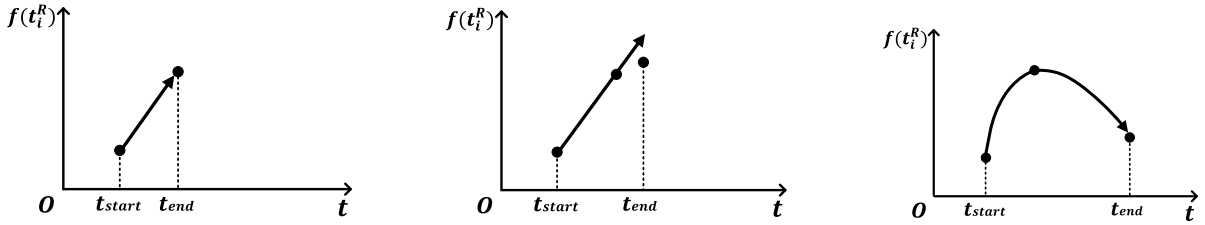
where n is the total number of original trajectory points.

IV. VOLTCom Algorithm

In this section, we describe the methodology of VOLTCom. First the implementation of the algorithm in this paper is described, followed by the implementation using pseudo-code and an explanation of the code description. Finally, we illustrate the core vector extraction algorithm with an enumeration of practical situations.

VOLTCom uses a combination of first-order and second-order Bézier functions and the given threshold to generate a function of the trajectory data with time first. Then the trajectory data vectors are extracted and represented according

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.



(a) Initial state (b) Vectors are not updated (c) Vector are updated

Fig. 2. Schematic diagram of the basic steps of VOLTCom algorithm execution.

to the defined vector features. Finally, the obtained vectors are organized in time series as the result of trajectory data compression. The specific steps of this algorithm are described as follows.

1) Receiving the sample points of the original trajectory T after cleaning and formatting processes. If it is the first sample point, it is marked as the initial key point.
2) If the second sampling point is received, the vector function $f(.)$ is generated through the point and the initial key point using the first-order Bézier (Equ.1 and 2). Constructing the vector to mark it as the current vector activity to upload into the vector sequence, as shown in Fig2(a); otherwise, performing Step.3.
3) The temporal property t_i of the received trajectory point p_i is substituted into the current vector activity $f(.)$, and calculate the positional coordinates (lat_i', lng_i') of the approximate trajectory point according to the current motion state (Equ.5,6,7 and 8).
4) Given an appropriate distance threshold α according to the accuracy requirements of the compression, the sed of $\widehat{p_i p_i'}$ is compared with α. If the $sed\left(\widehat{p_i p_i'}\right)$ is less than α, there is no need to generate a new vector function, just extend the current vector ends to t_i, as shown in Fig2(b); otherwise, perform Step.5.
5) If $sed\left(\widehat{p_i p_i'}\right)$ is greater than α, the corresponding point are calculated using the start and end times of the current vector. Constructing a vector function $f(.)$ from these points and the received trajectory point through a second-order Bézier function (Equ.1,3) and creating a new vector activity, as shown in Fig2(c). This vector is marked as the current vector and stored in the vector sequence.
6) Determining whether there are any subsequent trajectory points. If there are, repeat Step.3 to Step.5; otherwise, the vector sequence in time series is output as the compression result.

A. Algorithm 1implementation

The pseudo-code of the algorithm is given in Algorithms 1 and 2.

The Algorithm 1 describes the process of vector extraction of the GPS trajectory data. For each raw trajectory data point received and processed (Line 1), the first point is set as the initial point of the vector sequence (Lines 2 and 3). The first vector is generated using the first two points (Lines 4 and 5).

Algorithm 1 Vector Extraction – extractVector (T, α)

Input: T: Original trajectory sequence;
α: Vector update threshold.
Output: R: Compressed vector sequence.

```
1  for eaech p_i ∈ T do
2      if p_i is first then
3          p_i ← inital_point
4      else if p_i is second then
5          vector ← createVector(p_i, R)
6      else
7          vector_cur ← getCurVector(R)
8          p_i' ← getEstimate(p_i.t, vector_cur)
9          if sed(p_i p_i') < α then
10             vector_cur.t_end ← p_i.t
11         else
12             vecotor ← createVector(p_i, R)
13         end
14     end
15     save vector as vector_cur to R
16 end
17 return R
```

For a subsequent trajectory point, first, it is substituted into the current vector function (if the current vector function is the first-order Bézier function, it is substituted into Equ.5 and 6; if it is the second-order Bézier function, it is substituted into Equ.7 and 8) to find out the estimated position at that time (Lines 6 to 8). If the sed between two points is less than the threshold, the end time of the original vector is kept and set to that time (Lines 9 and 10). Otherwise, the vector function is recalculated and a new vector activity is generated (Lines 11 and 12). Finally, the changed vector saved as the current vector in the vector sequence (Lines 15 to 17).

The Algorithm 2 describes the process of vector generation. If the receiving point is the second trajectory point, the vector is generated using the initial point and that point to construct the first-order Bézier function by Equ.1 and 2(Lines 1 to 3). Otherwise, firstly, the current vector activity is obtained (Lines 4 and 5). Then, the start and end times is substituted into the vector function to obtain the coordinates of the start and end points (Line 6 and 7). After that, the vector function $f(.)$ is constructed by the second-order Bézier function (Equ.1 and 3) using the starting and ending points and the

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Vector Generation – createVector(p, R)

Input: p: Currently received trajectory point;
R: Currently stored vector sequence.
Output: $vector$: the newly generated vector

1 **if** p *is second* **then**
2 $f(t_i^R) \leftarrow first_Bézier(p, R.inital_point)$
3 $vector \leftarrow (f(t_i^R), inital_point.t, p.t)$
4 **else**
5 $vector_cur \leftarrow getCurVector(R)$
6 $p_{start} \leftarrow vector_cur.f(t_{start})$
7 $p_{end} \leftarrow vector_cur.f(t_{end})$
8 $f(t_i^R) \leftarrow second_Bézier(p_{start}, p_{end}, p)$
9 $vector \leftarrow (f(t_i^R), p_{end}.t, p.t)$
10 **end**
11 **return** $vector$

receiving points to generate a vector (Lines 8 and 9). Finally, the newly generated vector is returned (Line 11).

B. Example

In the real world, the trajectory of a moving object is not single, but contains multiple cases. Therefore, we use smart mobile devices with GPS positioning to collect spatio-temporal trajectory data of moving objects to illustrate the application of the method in this paper in different situations. The main purpose of vector extraction is to represent the sampled points in the form of vector features. When newly sampled data arrives, it is compared with the current vector activity. If the distance between the actual trajectory point position and the vector computed value is less than the specified threshold, it does not need to be processed; only when the motion state of the object changes so that the distance between the two is greater than the threshold, the vector features need to be regenerated and updated. In this way, vector extraction can effectively reduce the storage space of trajectory data.

In the actual vector extraction process, there are several cases as follows.

Case1: As shown in Fig3(a), when the object maintains an approximate uniform linear motion, only one vector feature needs to be saved to record the object's motion trajectory using the vector extraction method. Because the estimated position of the vector at the corresponding time of the object with almost constant motion state approximates the actual position, the vector is not updated. In this case, the vector extraction method embodies a great storage advantage.

Case2: As shown in Fig3(b),when the direction of the object remains the same and the velocity magnitude changes, the vector extraction algorithm generates a new vector feature adapted to the current velocity magnitude for vector update.

Case3: As shown in Fig3(c),when the velocity magnitude of the object remains constant but the direction of motion changes, it also leads to a large deviation of the vector calculated position from the actual position, generating a new vector feature and vector update.

In summary, vector features are generated and updated when the motion state of an object changes. The situation in the actual vector extraction process is more complex and may be a combination of the above. We can determine the change in the motion of the object based on the consistency of the vector features. In this way, we can apply this method to the recording and analysis of traffic trajectory data, which can record the effective information of trajectory data while occupying a small amount of space, and analyze the movement route and state change of the object by the change of vector function, and provide suggestions for various kinds of research such as user analysis and road condition analysis.

V. Experiments

In this section, we first describe the experimental setup and the dataset used. Then, we perform a large number of experiments to compare the proposed algorithm with the typical algorithms in various aspects of analysis.

A. Dataset And Experimental Setup

In this paper, the GPS trajectory data of Beijing taxis and the Geolife dataset from Microsoft Asia Research Institute [8] are used as the algorithm validation dataset. The Geolife dataset includes the GPS trajectory data of 182 users collected from April 2007 to August 2012, with a total of 17,621 trajectories, a total distance of 129,295 km, and a total duration of 50,176 h; the taxi dataset contains the trajectories of more than 100 vehicles in Beijing for two days, with a total of 719,816 trajectories. Since the dataset contains huge information and the entire content is not needed for this experiment, three subsets of users are randomly selected from Geolife as representatives,which are represented by Gl1, Gl2, and Gl3, where Gl1 contains 31830 trajectory points, Gl2 contains 8448 trajectory points, and Gl3 contains 43206 trajectory points with a sampling rate of 1-5s; the entire trajectory data of a vehicle taxi data in Beijing is selected as BJtaxi, and the experimental illustration of the compression effect is carried out with a sampling rate of 5s, and its GPS data between 23:24:42 on June 1, 2013 and 23:20:07 on June 3, 2013 were used for the rest of the comparison experiments, containing 983 trajectory points. To ensure the proper conduct of the experiment, we filtered the abnormal data caused by obstacles affecting the GPS readings and extracted valid data items such as position, time, and marker number.

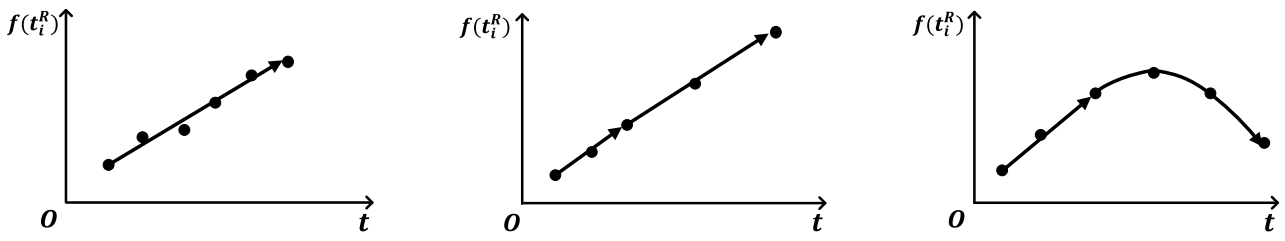
VOLTCom is implemented in python language; the IDE is JetBrains company Pycharm; the computer processor is Intel Core i7-7500U 2.70GHz; the physical memory is 8GB; the system type is 64-bit Windows10 operating system.

B. Experiment Results And Analysis

To evaluate the performance of the algorithm accurately and comprehensively, we use three evaluation indicators: compression ratio, compression error and execution time.

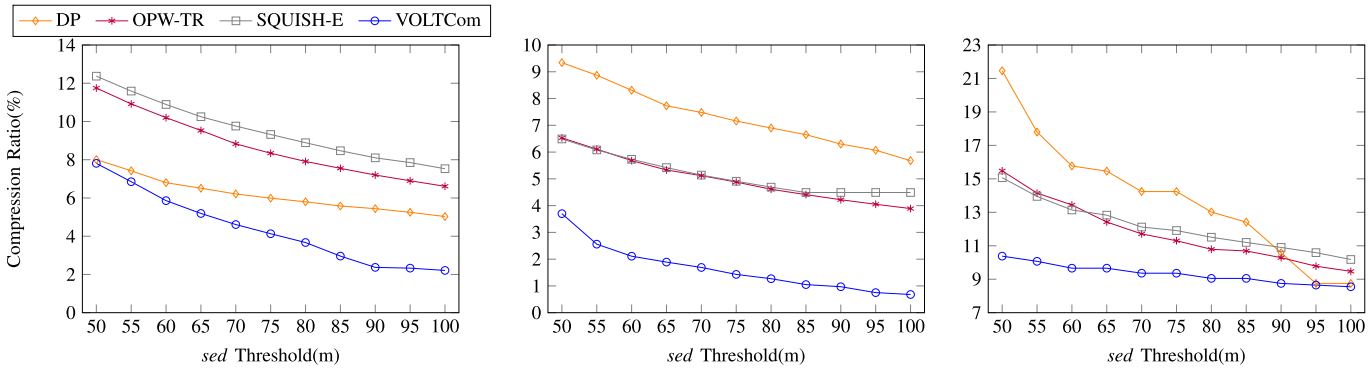
A total of 5 groups of experiments are carried out in this paper. The first three groups of experiments compare the performance of each algorithm in three aspects: compression ratio, compression error and execution time; the fourth group of experiments shows the compression effects of VOLTCom and analyze the relationships between vector density and

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.



(a) Case1:approximately uniform linear motion. (b) Case2: the magnitude of velocity changes. (c) Case3:the direction of motion changes.

Fig. 3. Schematic diagram of the basic steps of VOLTCom algorithm execution.



(a) Gl1 (b) Gl2 (c) BJtaxi

Fig. 4. Compression ratio of each algorithm under different datasets based on *sed* threshold.

velocity variation; the last set of experiments is designed to demonstrate that the algorithm outperforms the comparison algorithm in terms of average compression time on different orders of compressed data.

In order to more clearly demonstrate the superiority of this algorithm, the experiments are taken to be compared in a highly compressed scenario. As the compression level increases, the average value of the retained key trajectory points will increase, which can reduce the impact of the less information trajectory points on the experiment.

1) Compression Ratio: The compression ratio of trajectory is an important indicator to measure the effect of trajectory compression. The compression ratio calculation in this paper is shown in Def.3. This group of experiments compares the compression ratios of the four compression algorithms under different datasets by setting a distance threshold of 50m to 100m. The experimental results are shown in Fig. 4, and the experimental results are analyzed as follows.

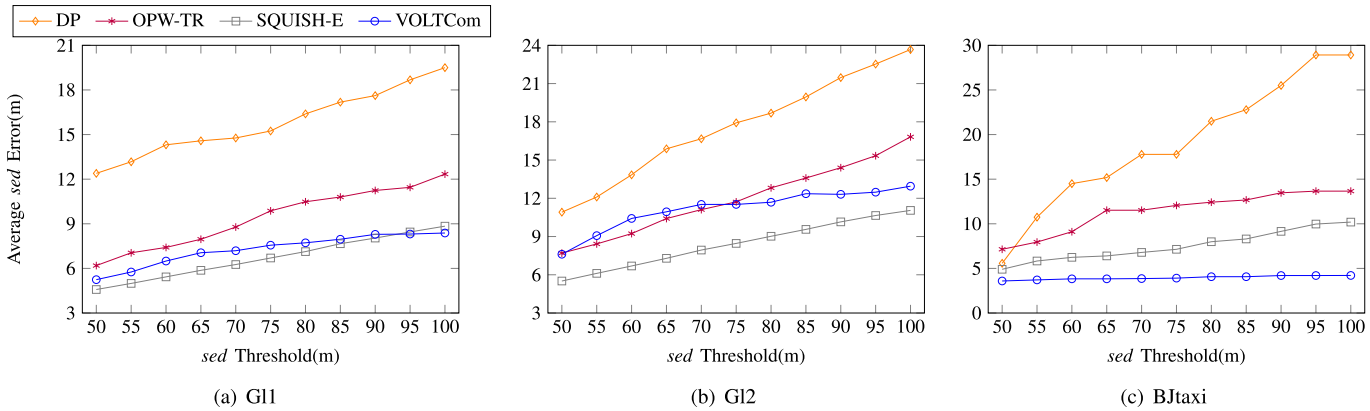
1) The data compression ratio of each algorithm generally decreases with the increase of the threshold value. The main reason is that the larger the selected threshold value is, the more difficult it is for the distance difference between the actual trajectory point and the corresponding compression trajectory point to exceed the threshold value. This results in fewer trajectory points that can be retained and the compression ratio decreases.
2) The compression ratios of VOLTCom outperform DP, OPW-TR, and SQUISH-E(*sed*)overall under three datasets. Among them, the compression ratio of DP varies greatly in three different datasets, and the compression ratios of OPW-TR and SQUISHE (*sed*) are relatively close.
3) In the BJtaxi dataset, the compression ratio of VOLTCom is less affected by the distance threshold. The data volume of the dataset is small, and the trajectory points are the main key points that are retained after being compressed to a certain degree. Therefore, the compression rate tends to be stable.

2) Compression Error: The error is one of the important criteria to measure the effectiveness of the trajectory compression fit. VOLTCom uses the average simultaneous Euclidean distance (Def.5) as the error metric. This group of experiments compares the errors of each algorithm under the distance threshold and compression threshold, respectively.

The experiments are conducted to compare the average errors of four compression algorithms under different datasets by setting the distance thresholds to from 50m to 100m. The experimental results are shown in Fig. 5:

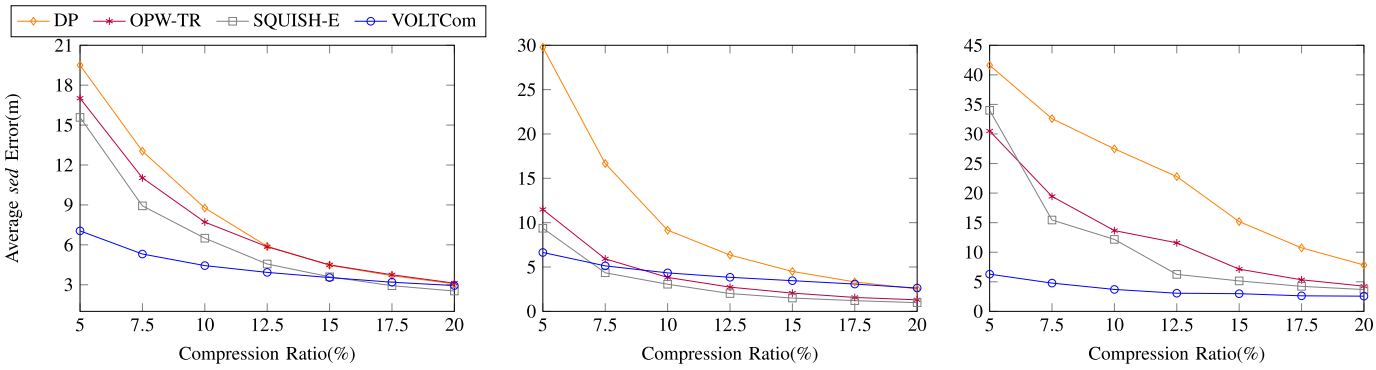
1) On the whole, the average distance error caused by each compression algorithm increases as the distance threshold increase. The increase in distance threshold leads to fewer retained trajectory points. As a result, the amount of information becomes less and the average error subsequently becomes larger.
2) In three different datasets, the average error of DP is much higher than the other three algorithms. The reason is that time is not considered in this algorithm. The vertical euclidean distance from the trajectory point to the compressed trajectory is directly used as the error to increase the average error.

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.



(a) Gl1 (b) Gl2 (c) BJtaxi

Fig. 5. Average error of each algorithm under different datasets based on *sed* thresholds.



(a) Gl1 (b) Gl2 (c) BJtaxi

Fig. 6. Average *sed* error of each algorithm under different datasets based on compression ratio.

3) VOLTCom has great advantages in terms of compression errors in the three datasets. In the dataset of Gl1 and Gl2, the average error of VOLTCom is second only to SQUISH-E (*sed*). And it can be seen from the figure that with the increase of distance threshold, the average error difference between the two algorithms keeps closing, and even VOLTCom has the trend of being lower than SQUISH-E(*sed*). In the BJtaxi, the difference of the four algorithms is obvious, and VOLTCom reflects the greater superiority. According to the average distance error from small to large, the four algorithms in order are VOLTCom, SQUISH-E (*sed*), OPW-TR and DP.

To further verify the performance of the proposed algorithm in compression accuracy, we adjust the distance threshold of the algorithm, so that each algorithm can achieve the compression ratio of 5% to 20% in the three datasets. The compression errors of the four algorithms at the same compression ratio are compared. The experimental results are shown in Fig. 6:

1) Overall, the average error of the four algorithms increases as the compression ratio decreases. The increase in compression leads to the loss of more information and an increase in the average error.
2) In different datasets, when the compression ratio is 7.5%, the average error growth rate of the other three algorithms, except for VOLTCom, increases significantly. With the increase of the compression degree, when the low compression rate reaches a certain limit, the more critical trajectory points will be discarded to ensure the compression ratio, which increases the error. VOLTCom shows strong stability in this case. The main reason is that the algorithm uses the vector function access, using Bézier curve calculation, which has better fitting effects than that of the straight lines.
3) In the three datasets, the average error of VOLTCom is generally lower than the other three algorithms. The average error of DP is higher overall, and the average errors of OPW-TR and SQUISH-E are closer. These are particularly evidents in the BJtaxi.

3) Execution Time: Similar to the above experiments on average error, this group of experiments also sets two parameters, distance threshold and compression ratio, to compare the execution time of four algorithms.

The results of the execution time of each algorithm based on the distance threshold are shown in TableI,II and III. The experimental results show that:

1) According to the execution time, the algorithms are ranked as follows. VOLTCom < OPW-TR < DP < SQUISH-E. However, in the BJtaxi, the proposed algorithm in this paper is slightly lower than the OPW-TR due to the small amount of data.
2) In the three datasets,the execution time of SQUISH-E is much longer compared to the other three algorithms.

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

TABLE I
EXECUTION TIME(MS) OF ALGORITHMS BASED ON SED THRESHOLD(M) IN GL2

	DP	OPW-TR	SQUISH-E	VOLTCom
$sed = 50$	6,710	1,767	1,952,364	1,183
$sed = 55$	6,585	1,840	1,960,036	1,176
$sed = 60$	6,615	1,963	1,936,676	1,161
$sed = 65$	6,455	2,046	2,074,739	1,170
$sed = 70$	6,632	2,202	1,939,847	1,178
$sed = 75$	6,743	2,352	2,030,974	1,202
$sed = 80$	6,385	2,417	1,946,952	1,181
$sed = 85$	6,517	2,659	2,032,983	1,181
$sed = 90$	6,224	2,637	2,085,981	1,163
$sed = 95$	6,301	2,816	2,045,832	1,146
$sed = 100$	6,635	2,982	1,952,196	1,176

TABLE II
EXECUTION TIME(MS) OF ALGORITHMS BASED ON SED THRESHOLD(M) IN GL1

	DP	OPW-TR	SQUISH-E	VOLTCom
$sed = 50$	5,122	745	1,074,405	949
$sed = 55$	4,987	864	1,062,140	862
$sed = 60$	5,014	859	1,111,705	844
$sed = 65$	5,172	899	1,081,880	860
$sed = 70$	4,878	1,044	1,071,913	836
$sed = 75$	4,837	1,067	1,067,837	875
$sed = 80$	5,413	1,147	1,101,499	868
$sed = 85$	4,809	1,258	1,061,594	842
$sed = 90$	5,673	1,371	1,076,643	841
$sed = 95$	4,685	1,430	1,115,803	880
$sed = 100$	4,721	1,059	1,195,767	873

TABLE III
EXECUTION TIME(MS) OF ALGORITHMS BASED ON SED THRESHOLD(M) IN BJTAXI

	DP	OPW-TR	SQUISH-E	VOLTCom
$sed = 50$	107	16	1,134	33
$sed = 55$	98	17	1,724	28
$sed = 60$	99	19	1,132	28
$sed = 65$	96	21	1,312	33
$sed = 70$	95	15	1,283	27
$sed = 75$	96	17	1,261	28
$sed = 80$	92	18	1,189	28
$sed = 85$	89	18	1,188	35
$sed = 90$	91	20	1,021	27
$sed = 95$	78	13	1,149	29
$sed = 100$	80	28	1,052	28

The execution time of the DP algorithm is relatively stable. The execution times of VOLTCom and OPW-TR are relatively close.

The experimental results of the execution time of each algorithm based on the compression ratio are shown in Table.IV,V and VI. Experimental results show that:

1) Similar to the results of distance threshold-based experiments. The longest execution time is still SQUISH-E, followed by DP. When the compression rate is around 10%, the execution times of our algorithm and OPW-TR converge. The execution time of OPW-TR is smaller before that, and after that, the execution time of VOLTCom is the smallest.

TABLE IV
EXECUTION TIME(MS) OF ALGORITHMS BASED ON COMPRESSION RATIO(%) IN GL2

	DP	OPW-TR	SQUISH-E	VOLTCom
$ratio = 5.0$	6,395	2,207	96,931	1,211
$ratio = 7.5$	6,908	1,516	377,988	1,177
$ratio = 10.0$	7,351	1,063	178,126	1,150
$ratio = 12.5$	7,475	845	363,713	1,137
$ratio = 15.0$	7,641	679	349,358	1,212
$ratio = 17.5$	9,575	570	339,656	1,166
$ratio = 20.0$	7,575	496	326,321	1,099

TABLE V
EXECUTION TIME(MS) OF ALGORITHMS BASED ON COMPRESSION RATIO(%) IN GL1

	DP	OPW-TR	SQUISH-E	VOLTCom
$ratio = 5.0$	4,950	2,124	51,068	873
$ratio = 7.5$	5,283	1,251	101,853	1,071
$ratio = 10.0$	5,274	1,036	101,501	843
$ratio = 12.5$	5,701	864	198,267	828
$ratio = 15.0$	5,961	558	192,168	846
$ratio = 17.5$	5,776	429	184,435	832
$ratio = 20.0$	5,950	356	172,957	820

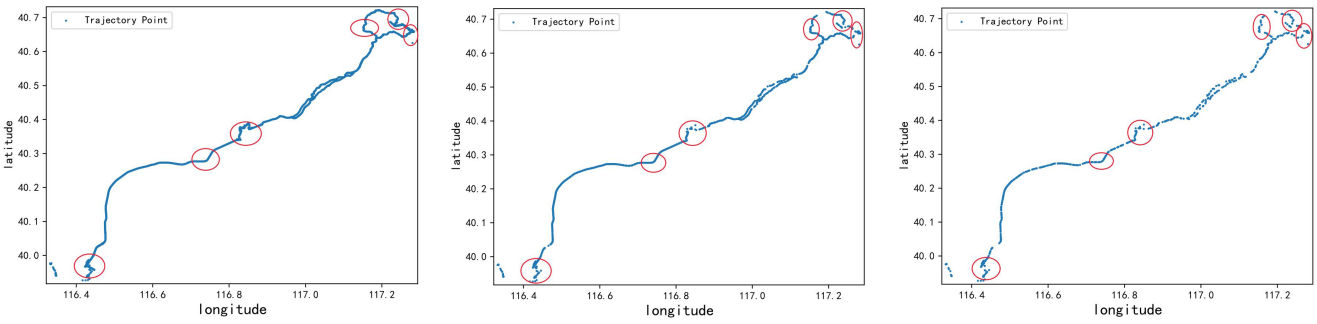
TABLE VI
EXECUTION TIME(MS) OF ALGORITHMS BASED ON COMPRESSION RATIO(%) IN BJTAXI

	DP	OPW-TR	SQUISH-E	VOLTCom
$ratio = 5.0$	72	37	74	25
$ratio = 7.5$	71	29	200	25
$ratio = 10.0$	89	18	114	25
$ratio = 12.5$	100	21	174	26
$ratio = 15.0$	105	16	172	34
$ratio = 17.5$	103	12	195	28
$ratio = 20.0$	103	12	197	26

2) As the compression ratio decreases, the execution time of OPW-TR increases gradually. When the compression ratio is reduced, the 'window' needs to be enlarged, so that more trajectory points fall into the 'window'. When a new point is added, the algorithm will recalculate the *sed* of each point in the window. So the number of computations increases and the execution time of the algorithm goes up.
3) The execution time of VOLTCom is relatively stable under different datasets and compression ratios. The reason is that the algorithm only needs to substitute the previous vector function to calculate the predicted coordinates for the newly added trajectory points and determine whether to generate a new vector. No deep iterations and complex calculations are required. This greatly reduces the time complexity and ensures the stability and superiority of the algorithm execution time.

4) Compression Effect and Relationship : To visually illustrate the compression effects of VOLTCom, the trajectory data of Gl3 are now compressed using the proposed algorithm. The distance thresholds are 50m and 100m, and the compression ratios are 10.59% and 6.76%, respectively. Although the storage of VOLTCom uses a vector sequence, it is mainly

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.



(a) Original Trajectory (b) $sed = 50$m (c) $sed = 100$m

Fig. 7. VOLTCom compression effect under different *sed* thresholds.

TABLE VII
AVERAGE EXECUTION TIME(MS) OF ALGORITHMS IN GL1

	DP	OPW-TR	SQUISH-E	VOLTCom
$size = 10$	0	0	0	0
$size = 50$	1	1	3	1
$size = 100$	1.5	4	11	3
$size = 500$	15	16	298	14
$size = 1{,}000$	60	24	1,064	26
$size = 5{,}000$	438	121	26,291	125
$size = 10{,}000$	1,310	464	101,671	250

composed of vector functions with time as the variable. That is, the coordinates of the corresponding compressed trajectory points can be derived from the key time points. The original trajectory and the compressed trajectory are shown in Fig.7:

1) In the figure, the red circles are the detail comparison parts of each trajectory image. It can be seen that as the threshold value increases, the number of compressed trajectory points decreases. But it does not affect the preservation of the original trajectory shape features. At the same time, due to the storage characteristics of the vector function, the proposed algorithm can also appropriately increase the key time points according to the compression requirements and describe the original trajectory in more detail by calculating the corresponding position.
2) The compression results of this algorithm can also reflect the speed change fast or slow by the density of vectors. When the velocity of the object changes and the previously stored vector function cannot describe the corresponding position under the changed velocity, it is necessary to construct a new vector function to generate new vectors. Therefore, the faster the speed of the object changes in an equal time, the more vectors are generated and the vector density increases in that period.

5) Time Superiority : To further demonstrate the superiority of the time complexity of VOTCom, we extracted different orders of magnitude of trajectory points from the Gl1 user dataset and compressed them using four trajectory compression algorithms. The total execution time (Ms) of each algorithm is shown in the following TableVII.

The average execution time of each algorithm is shown in Fig. 8. As can be seen from the figure, when the number of extracted trajectory points is less than 500, the execution

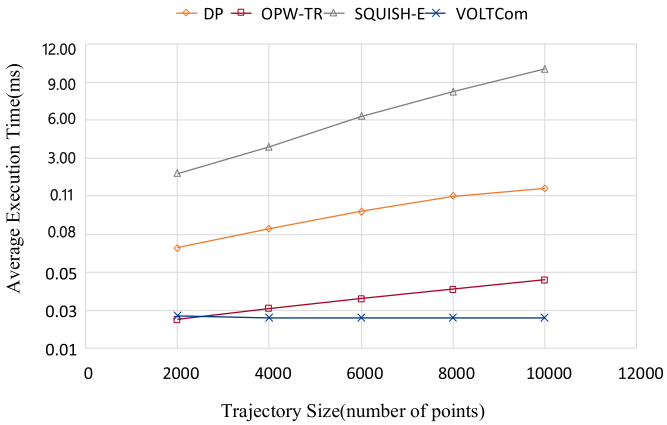


Fig. 8. Average execution time of algorithms at different trajectory scales.

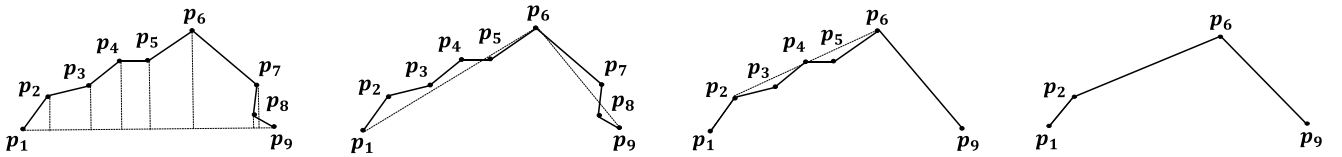
time of each algorithm is very short because the experimental data are too small, so the execution time between different algorithms cannot be well separated. When the number of track points is greater than 500, the average execution times of DP, OPW-TR and SQUISH-E increase with the number of trajectory points, and approximate a sloping straight line on the image; In contrast, the average execution time of VOLTCom is stable, independent of the amount of data, and appears as a horizontal straight line on the image. The image of SQUISH-E is slightly deviated from the theory due to the influence of data selection and CPU scheduling execution. The linear time complexity of VOLTCom gives it a shorter time advantage for trajectory compression on larger datasets.

VI. COMPARATIVE ALGORITHM AND COMPLEXITY ANALYSIS

In this section, we explain the reasons for choosing OPWTR, DP and SQUISH-E as the comparison algorithms. The time complexity of the different algorithms is also analyzed theoretically from the perspective of the algorithm execution process to verify the time superiority of VOTCom in the above experiments.

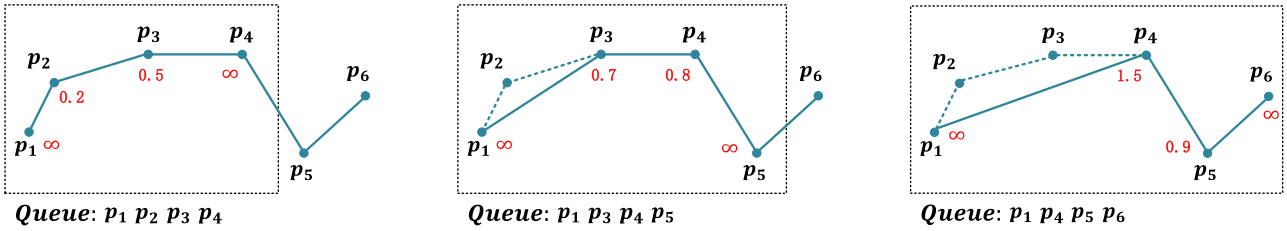
OPW-TR, DP and SQUISH-E are used as comparison algorithms of VOLTCom in this paper. DP and OPW-TR represent the classical offline and online trajectory compression algorithms, respectively as references. SQUISH-E can set

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.



(a) Original trajectory (b) p_6 is the farthest and start recursion (c) Partial approximation using $\widehat{p_6p_9}$ (d) Partial approximation using $\widehat{p_2p_6}$

Fig. 9. Execution of Douglas-Peucker: The original trajectory is recursively approximated by selecting the point farthest from each line segment until that distance is below a user-specified tolerance. The dashed lines are auxiliary lines, and the black dots and solid lines represent a compressed representation of that trajectory.



(a) Enqueuing p_1, p_2, p_3 and p_4 (b) Dequeuing p_2 and enqueuing p_5 (c) Dequeuing p_3 and enqueuing p_6

Fig. 10. Execution of SQUISH-E: The estimated sed error is used as the priority for each point and the point that is expected to introduce the lowest error in the compression process is removed. The blue dots and solid lines are the compressed representation of the trajectory, the red numbers represent the priority of each point, and the dashed rectangular boxes indicate the points that have been currently processed.

two parameters of compression ratio and distance threshold. When the ratio is not set to 1, it is the online compression method, and in the other cases it is the offline compression method. This enables the experiment to highlight the characteristics of the algorithm more comprehensively from different perspectives. Therefore, the SQUISH-E is also chosen as a comparison algorithm.

The implementation of DP uses a recursive approach. First, the starting and ending points of the trajectory is connected; the vertical distance from the point in between to the line is calculated, and it is divided into two sub-tracks at the midpoint of the maximum vertical distance; Then, the sub-trajectories are continuously divided until the maximum vertical distance is less than a given threshold. As shown in Fig.9 (a), the vertical distance from p_2-p_8 to line segment $\widehat{p_1p_9}$ is calculated. Among them, the vertical distance corresponding to p_6 is the largest and exceeds the given threshold, so the trajectory is divided from p_6 into two parts, $\widehat{p_1p_6}$ and $\widehat{p_6p_9}$, as shown in Fig. 9 (b). The above process is repeated for $\widehat{p_1p_6}$ and $\widehat{p_6p_9}$, respectively, until the maximum vertical distance is less than the threshold. Finally, the start and end points of the segment is kept as shown in Fig.9 (c) (d). Thus, the time complexity of the DP algorithm is $O\left(n^2\right)$.

OPW-TR uses a valid line segment to fit points in a growing sliding window. Each time the length of the sliding window is increased, the *sed* from the trajectory point in the window to the fitted line segment is obtained. The point with the largest error is used as the closing point of the approximating line segment and the starting point of the next segment. As shown in Fig.11, the first point p_1 of the trajectory is initialized as the anchor point, and then the sliding window starts to increase. When the window contains p_1, p_2, and p_3, none of the synchronous Euclidean distances between the fitted

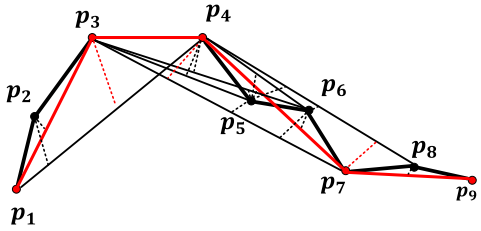


Fig. 11. Execution of OPW-TR:Using line segments to fit the trajectory points in the open window of growth. The black bolded solid lines and dots indicate the original trajectory, red dots and solid lines indicate the compressed trajectory. The red dashed line indicates that the SED exceeds a given threshold.

line segments and the original trajectory is greater than the specified error threshold. When the window contains p_4, the distance from p_3 to line segment $\widehat{p_1p_4}$ exceeds the threshold, so $\widehat{p_1p_3}$ is used as the part of the compressed trajectory and p_3 is set as the new anchor point. The above steps continue to be repeated. Thus, the time complexity of OPW-TR is $O\left(N^2\right)$.

SQUISH-E contains two parameters: threshold μ and the compression multiplier λ. It is based on a priority queue and calculates an upper bound on the *sed* error that will be introduced after deleting each point and takes it as the priority of that point. The compression is done by finding the point with the minimum priority to remove it each time. The minimum error compression is achieved with guaranteed compression multiplicity, after which the result is compressed again within the threshold value to achieve maximum compression. As shown in Fig. 10(a), the number of compressed trajectory points is determined to be 4 according to λ. The priority of the initial point p_1 is set to ∞. Assuming we delete p_2, the *sed* error introduced is 0.2, taking 0.2 as

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

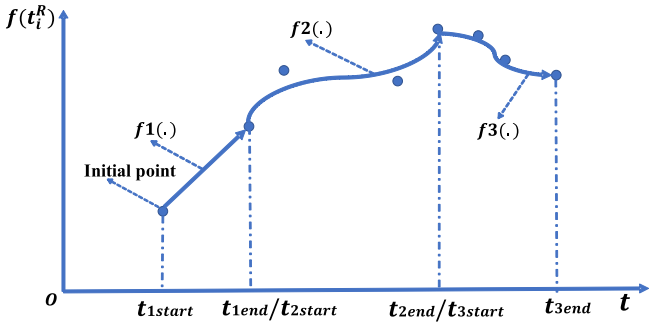


Fig. 12. Execution of VOLTCom:The compressed trajectory is represented by a sequence of vector features. $f(.)$ is generated by substituting key points into the Bézier function and is a specific function with time as the independent variable.

TABLE VIII
ALGORITHM DETAILS COMPARISON

	Type	Parameters	Time Complexity
DP	Offline	*sed*	$O(N^2)$
OPW-TR	Online	*sed*	$O(N^2)$
SQUISH-E(*sed*)	Offline	*sed*	$O(N \log N)$
SQUISH-E(ratio)	Online/Offline	ratio	$O(N \log \frac{N}{\lambda})$
VOLTCom	Online	*sed*	$O(N)$

the priority of p_2. Similarly, the priorities of p_3 and p_5 are 0.5 and 0.8, respectively. In Fig. 10 (b), when p_5 joins, in the priority queue $Queue$, p_2 has the smallest priority and is therefore the first to be deleted, p_1, p_3, p_4 and p_5 are left. With the addition of subsequent points, the trajectory points with the lowest priority in the queue are deleted in turn to achieve compression, as shown in Fig. 10(c). The time complexity in finding the trajectory points with minimum priority is $O(\log N)$. Therefore, the time complexity of this algorithm for N original trajectory points is $O(N \log N)$ under offline compression with the threshold μ and $O\left(N \log \frac{N}{\lambda}\right)$ under compression multiplicity λ.

VOLTCom is based on the vector generation and extraction algorithm to achieve trajectory compression. The vectors are generated by substituting the trajectory points into the first-order and second-order Bézier functions (Equ.2 and 3) and then stored. After that, the position of the next time point is predicted to decide whether to generate a new vector or not. As shown in the figure, the original trajectory is fitted by the functions $f1(.)$, $f2(.)$, $f3(.)$ and the corresponding start and end time points for each segment of the function. In this process, only one operation is needed for the trajectory points. Even if a new vector is to be generated, it is only constructed by linearly calculating the position coordinates of the existing previous vector, without recursive search and other operations. So the time complexity of the method is $O(N)$. This reflects the great superiority of the algorithm in time complexity.

Through the process analysis of the above algorithms, the details of each algorithm are summarized in the Table.VIII.

VII. CONCLUSION

This paper focuses on the problem that existing trajectory compression algorithms use key points to store the compression results, which leads to the problem that they do not reflect the trajectory continuity in time and object velocity variation well. We propose an online trajectory compression algorithm based on vector extraction. The algorithm generates vector functions in real-time through Bézier curves and extracts constructed vector features. A vector sequence is used to represent and store the compressed trajectory. The continuity of the vector function over time and the density variation of the vector can theoretically solve the above problem. At the same time, the method has a linear time complexity through the process analysis and verification of the algorithm. It is significantly better than the comparison algorithms in terms of execution time when the data magnitude is larger. In addition, experiments based on real datasets prove that VOLTCom also has advantages over comparative compression algorithms in terms of compression rate and error, which is meaningful for the storage and analysis of traffic data.

In the future, the algorithm will continue to be improved from the following aspects.

- Other factors that can reduce the error, such as direction angle, instantaneous speed, etc. are added to the compression calculation process to improve the accuracy of the algorithm;
- Seeking a better function to carve the compression trajectory for objects with frequent velocity changes to ensure the effectiveness of the algorithm. Avoid frequent velocity changes leading to too dense vectors;
- Using techniques such as data mining to explore more valuable information in trajectories, we consider whether the method can be extended to other aspects such as trajectory similarity measurement to improve the usefulness of the algorithm [23], [24], [25].

REFERENCES

[1] S. Mcmurray and A. H. Sodhro, "A study on ML-based software defect detection for security traceability in smart healthcare applications," *Sensors*, vol. 23, no. 7, p. 3470, Mar. 2023.
[2] M. Ali, L. Tang Jung, A. Hassan Sodhro, A. Ali Laghari, S. Birahim Belhaouari, and Z. Gillani, "A confidentiality-based data Classification-as-a-Service (C2aaS) for cloud security," *Alexandria Eng. J.*, vol. 64, pp. 749–760, Feb. 2023.
[3] N. A. Zardari, R. Ngah, O. Hayat, and A. H. Sodhro, "Adaptive mobility-aware and reliable routing protocols for healthcare vehicular network," *Math. Biosci. Eng.*, vol. 19, no. 7, pp. 7156–7177, May 2022.
[4] A. Lakhan et al., "Dynamic application partitioning and task-scheduling secure schemes for biosensor healthcare workload in mobile edge cloud," *Electronics*, vol. 10, no. 22, p. 2797, Nov. 2021.
[5] Z. Yi-Meng, L. Xue-Lian, M. Shuai, and J. Jia-Hao, "Road network aware online trajectory compression," *J. Softw.*, vol. 29, no. 3, pp. 734–755, 2018.
[6] F. Giannotti et al., "Unveiling the complexity of human mobility by querying and mining massive trajectory data," *VLDB J.*, vol. 20, no. 5, pp. 695–719, Oct. 2011.
[7] Y. Zheng, L. Zhang, X. Xie, and W.-Y. Ma, "Mining interesting locations and travel sequences from GPS trajectories," in *Proc. 18th Int. Conf. World Wide Web*, Apr. 2009, pp. 791–800.
[8] X. Lin, S. Ma, J. Jiang, Y. Hou, and T. Wo, "Error bounded line simplification algorithms for trajectory compression: An experimental evaluation," *ACM Trans. Database Syst.*, vol. 46, no. 3, pp. 1–44, Sep. 2021.
[9] Y. Jing, "Retrieval, mining and application of large scale trajectory data," Ph.D. dissertation, School Comput. Sci. Technol., Univ. Sci. Technol. China, Anhui, China, 2012.
[10] L. Li, X. Xia, and Z. Xiong, "A novel online trajectory compression algorithm for real-time trajectory surveillance applications," in *Proc. IEEE 3rd Adv. Inf. Manage., Communicates, Electron. Autom. Control Conf. (IMCEC)*, Oct. 2019, pp. 995–999.

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.

[11] P. Zhao, Q. Zhao, C. Zhang, G. Su, Q. Zhang, and W. Rao, "CLEAN: Frequent pattern-based trajectory compression and computation on road networks," *China Commun.*, vol. 17, no. 5, pp. 119–136, May 2020.
[12] C. Chen, Y. Ding, S. Guo, and Y. Wang, "DAVT: An error-bounded vehicle trajectory data representation and compression framework," *IEEE Trans. Veh. Technol.*, vol. 69, no. 10, pp. 10606–10618, Oct. 2020.
[13] D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartograph., Int. J. Geograph. Inf. Geovisualization*, vol. 10, no. 2, pp. 112–122, Dec. 1973.
[14] N. Meratnia and R. A. de By, "Spatiotemporal compression techniques for moving point objects," in *Proc. Int. Conf. Extending Database Technol.* Berlin, Germany: Springer, 2004, pp. 765–782.
[15] E. Keogh, S. Chu, D. Hart, and M. Pazzani, "An online algorithm for segmenting time series," in *Proc. IEEE Int. Conf. Data Mining*, Nov./Dec. 2001, pp. 289–296.
[16] J. Muckell, J.-H. Hwang, V. Patil, C. T. Lawson, F. Ping, and S. Ravi, "SQUISH: An online approach for GPS trajectory compression," in *Proc. 2nd Int. Conf. Comput. Geospatial Res. Appl.*, 2011, pp. 1–8.
[17] J. Muckell, P. W. Olsen, J.-H. Hwang, C. T. Lawson, and S. S. Ravi, "Compression of trajectory data: A comprehensive evaluation and new approach," *GeoInformatica*, vol. 18, no. 3, pp. 435–460, Jul. 2014.
[18] J. E. Hershberger and J. Snoeyink, "Speeding up the Douglas–Peucker line-simplification algorithm," Dept. Comput. Sci., Univ. British Columbia, Vancouver, BC, Canada, Tech. Rep., 1992.
[19] D. Zhang, M. Ding, D. Yang, Y. Liu, J. Fan, and H. T. Shen, "Trajectory simplification: An experimental study and quality analysis," *Proc. VLDB Endowment*, vol. 11, no. 9, pp. 934–946, May 2018.
[20] F. Qingfu, Z. Lei, L. Leijun, B. Suning, and F. Chen, "Online GPS trajectory data compression based on offset calculation," *Comput. Eng. Appl.*, vol. 53, no. 8, pp. 254–259, 2017.
[21] R. Fu-Jie, "Key technology research and typical application of vector extraction based on sensory data," M.S. thesis, School Comput. Sci., Fac. Inform., Beijing Univ. Technol., Beijing, China, 2018.
[22] J. Jun-wen and W. Xiao-ling, "Review on trajectory data compression," *J. East China Normal Univ. Natural Sci.*, no. 5, pp. 61–76, Sep. 2015.
[23] H. Zhang et al., "Trajectory similarity learning with auxiliary supervision and optimal matching," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, vol. 2015, 2021, pp. 3209–3215.
[24] P. Yang, H. Wang, Y. Zhang, L. Qin, W. Zhang, and X. Lin, "T3S: Effective representation learning for trajectory similarity computation," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Apr. 2021, pp. 2183–2188.
[25] C. Feng, Z. Pan, J. Fang, J. Xu, P. Zhao, and L. Zhao, "Aries: Accurate metric-based representation learning for fast top-k trajectory similarity query," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 499–508.



Zhi Cai received the M.Sc. degree from the School of Computer Science, The University of Manchester, in 2007, and the Ph.D. degree from the Department of Computing and Mathematics, Manchester Metropolitan University, U.K., in 2011. He is currently an Associate Professor with the College of Computer Science, Beijing University of Technology, China. His research interests include information retrieval, ranking in relational databases, keyword search, and intelligent transportation systems.



Qian Dong received the bachelor's degree from the School of Software, Hebei Normal University, in 2021. She is currently pursuing the M.Sc. degree with the College of Computer Science, Beijing University of Technology. Her main research interests include intelligent transportation systems and data mining.



Meihui Shi received the M.Sc. and Ph.D. degrees from the School of Computer Science and Engineering, Northeastern University, in 2017 and 2022, respectively. She is currently a Lecturer with the College of Computer Science, Beijing University of Technology, China. Her research interests include spatial-temporal data processing, recommendation systems, and data mining.



Xing Su received the B.Sc. degree from the School of Software Engineering, Beijing University of Technology, in 2007, and the M.Sc. and Ph.D. degrees in computer science from the University of Wollongong, Australia, in 2012 and 2015, respectively. He is currently a Lecturer with the Faculty of Informatics, Beijing University of Technology, Beijing, China. His research interests include distributed artificial intelligence, multi-agent systems, disaster management, and service-oriented computing.



Limin Guo received the bachelor's degree from the Huazhong University of Science and Technology in 2005 and the Ph.D. degree from the Institute of Software, Chinese Academy of Sciences, in 2012. She is currently a Lecturer with the Beijing University of Technology. Her research interests include database research and implementation and spatial-temporal data mining.



Zhiming Ding received the bachelor's degree from Wuhan University in 1989, the master's degree from the Beijing University of Technology in 1996, and the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, in 2002. He is currently a Professor with the College of Computer Science, Beijing University of Technology, China. His main research interests include database systems and mobile and spatial-temporal data management.

Authorized licensed use limited to: Fuzhou University. Downloaded on May 09,2026 at 08:30:02 UTC from IEEE Xplore. Restrictions apply.